

Documentation Hangman

Linus Kubis

October 2025

1 Summary

In this project, I created two versions of the well-known game Hangman. The first version is completely playable inside the console, while the other version was made with Pygame.

1.1 Console version

For the version inside the console, the user can decide whether they want to add custom words to the game. If he decides to add custom words, we will be asked to provide a list of words separated by a comma. Then, a new question will appear, asking the user to choose a category of words. If the user didn't add any new custom words, the file will be either empty or store the words from the previous game. To choose a wordlist, the user needs to type the respective number. After choosing a wordlist, the game will start. A random word gets selected, and the user gets displayed an ASCII picture of hangman, as well as the letters they guessed, the wrong letters, and the blank word (underscores). From there, the user needs to guess the word while not trying to complete the hangman picture. If they achieve it before Hangman is complete, then they win; otherwise, the user loses.

1.2 Pygame

The Pygame version functions similarly; however, after executing the code, the user is shown a menu where they need to decide whether to choose a wordlist or quit the game. When selecting the button "Choose Word List," the user gets displayed the different categories (which could be expanded by the user by adding new .txt files to the folder). The user can either choose an existing word list or click the button "Add custom words", where a new submenu opens. There, the user can type in new words and click save to write them to the file. In the "Choose Word List" menu, when the user decides on a list, the game starts. On the screen, on the left, the user can select the different letters by clicking on them (they will get deselected afterwards). On the right, Hangman and the blank word are displayed. From this, the logic is the same as in the other version.

2 Example use of the project requirements

I will use the console version of the game to show the requirements. I talked with the teacher about this, and it is possible to do it in this way, as the Pygame version does not include `print()` and `input()`.

- Comparisons and logical operations: In the function *choosewordlist*, comparisons and the logical operation AND are used to check if the number the user selected corresponds to the number of files in the directory. (*functions.py* line 29)

- *if* statements (with *elif* and *else*): For example, it is used to check if a guessed letter by the user is a letter in the word. With *elif*, it is checked whether the letter was already guessed or is not already in *wrongletters*. (*main.py* lines 71-83)
- Lists and at least 3 list operations: The project features several lists and list operations, such as *.split* to split the custom words from the user into different elements (*main.py* line 25). Or operations like *.append* (*main.py* line 73) and *.join* (*main.py* lines 61-62).
- *for* and *while* loops: A *while* loop is used for the main gameplay loop, to check if the user still has enough lives (*main.py* line 56). A *for* loop is, for instance, used when the blank word is created to count the number of letters (*functions.py* line 46)
- Calls for *input()* and *print()* functions: Throughout the program, there exist many uses of the *print()* (i.e. *main.py* line 19) and *input()* (i.e. *main.py* line 22) functions, to engage with the user.
- *try* statements: A *try* statement is used to check if the selected files can be opened to be read. When it is not possible, because the selected file does not exist, an error will be printed. (*main.py* line 40)
- Typecasting between different types: For instance, a typecast is used to typecast the number the user selects when choosing a word list into an integer. (*functions.py* line 26)
- Functions: The program features several functions in a separate Python file *functions.py*. For instance, the function *createblank* (line 44) is used to create an underscored word as long as the random word.
- Read and Write from files: Mostly the word files are being read by the program to access the words from the lists (i.e. *main.py* lines 41-42). And when the user adds custom words, these are written into a file (*main.py* line 28)

3 Acknowledgements

- Implementation of a filtered list of files in a directory: <https://stackoverflow.com/questions/2225564/get-a-filtered-list-of-files-in-a-directory>
- Hangman ASCII pictures: <https://gist.github.com/chrishorton/8510732aa9a80a03c829b09f12e20d9c>
- Hangman pictures: <https://github.com/techwithtim>
- Code for the button display: <https://github.com/adislksn/Hangman/blob/main/Hangman-code.py>

4 Authorship declaration

I declare that the work submitted here is from my authorship only. I haven't used any generative AI to help with any code/text included in my work. I have given credit for the help I had conceptualizing my project. My work respects the university and course code of conduct